Arhitektura računara i operativni sistemi

Laboratorijske vežbe 5

1. Niti

Posmatrajte sledeći kod napisan u programskom jeziku C:

```c
#include <stdio.h>

void myTurn(){

    while(1){
        printf("My turn!\n");
        sleep(1);
    }
}

void yourTurn(){

    while(1){
        printf("Your turn!\n");
        sleep(2);
    }
}

int main(){

    myTurn();
    yourTurn();
}
```

Slika 1 - Program bez niti (noThreads.c)

Rezultat izvršavanja ovog programa je sledeći:

```
My turn!
My turn!
My turn!
My turn!
My turn!
My turn!
My turn!
My turn!
My turn!
```

Slika 2 - Izvršavanje programa na slici 1

Ispisivaće se „My turn!“ zauvek, odnosno funkcija **`yourTurn()`** nikada neće doći na red za izvršavanje. Međutim, postoji mogućnost da se ovaj proces podeli na delove i to tako da se ove dve funkcije izvršavaju paralelno ili makar prividno paralelno.

U proces ćemo uvesti jednu nit. Ovoj niti ćemo dodeliti jedan posao, a to je da izvršava funkciju **`myTurn()`**. Da bismo kreirali nit, potrebno je da od kernela operativnog sistema zatražimo da kreira novu nit.

2. POSIX API

U ovom slučaju ćemo sa kernelom komunicirati pozivajući funkcije **POSIX API**-a (engl. *Portable Operating Systems Based on UNIX*), koje se su implementirane u C biblioteci **`pthread`**.

API (engl. *Application Programming Interface*) nije pojam vezan samo za ovu oblast, a predstavlja set funkcionalnosti koje su nam dostupne za korišćenje, odnosno, izložene od strane nekog sistema. Do sada ste koristili, recimo, *Java API*, pa ste mogli da pozivate različite metode u *Javi* (na primer metoda koja proverava dužinu stringa, metoda koja dodaje element u listu itd.). Postoje i druge vrste API-ja, poput web API-ja, gde su nam od strane serverske aplikacije izložene funkcionalnosti koje može pozivati klijentska aplikacija. POSIX API nam nudi različite funkcije za rad sa operativnim sistemom, između ostalog i sa nitima operativnog sistema, procesima, interprocesnom komunikacijom i slično.

Koristićemo funkcije POSIX API-ja kako bismo kreirali niti, mutekse, semafore i manipulisali njima. U naš program smo uključili biblioteku **`pthread.h`**. U *main* funkciji deklarišemo novu promenljivu i to tipa **`pthread_t`**. Pomoću ove promenljive ćemo referencirati nit koju inicijalizujemo pomoću funkcije **`pthread_create`**. Ova funkcija prima adresu promenljive, pokazivač na strukturu atributa niti, adresu funkcije koju nit treba da izvršava i argument koji se prosleđuje toj funkciji. Funkcija koju prosleđujemo niti kao zadatak koji treba da izvršava vraća rezultat tipa **`void*`** i prima argument tipa **`void*`**. Ovakav tip funkcije nam pruža mogućnost da konvertujemo povratnu vrednost funkcije, odnosno, argument funkcije u bilo koji drugi tip. Na ovaj način funkcija **`pthread_create`** ima najopštiji oblik. Atributi koje koje nit može imati predstavljeni su strukturom **`pthread_attr_t`** u kojoj se mogu konfigurisati adresa steka niti, veličina steka, i druge osobine.

Kroz primere ćemo koristiti i druge funkcije POSIX API-ja, a neke od njih su **pthread_join**, **pthread_mutex_lock**, **pthread_mutex_unlock**, **sem_init**, **sem_post**, **sem_wait**, kao i tipove podataka **pthread_t**, **pthread_mutex_t**, **sem_t** i druge.

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start_routine) (void *), void *arg);
```

Slika 3 - Funkcija pthread_create

```
#include <stdio.h>
#include <pthread.h>

void* myTurn(void *arg){

    char* msg = (char*)arg;

    while(1){
        printf("%s\n", msg);
        sleep(1);
    }
}

void yourTurn(){

    while(1){
        printf("Your turn!\n");
        sleep(2);
    }
}

int main(){

    pthread_t thread;
    char* msg = "My turn!";

    pthread_create(&thread, NULL, myTurn ,(void*)msg);

    //myTurn();
    yourTurn();
}
```

Slika 4 - Inicijalizacija niti u programu (threads.c)

3. Promena konteksta

Izmenjeni fajl se ponovo kompajlira i kada se pokrene, možemo uočiti promene:

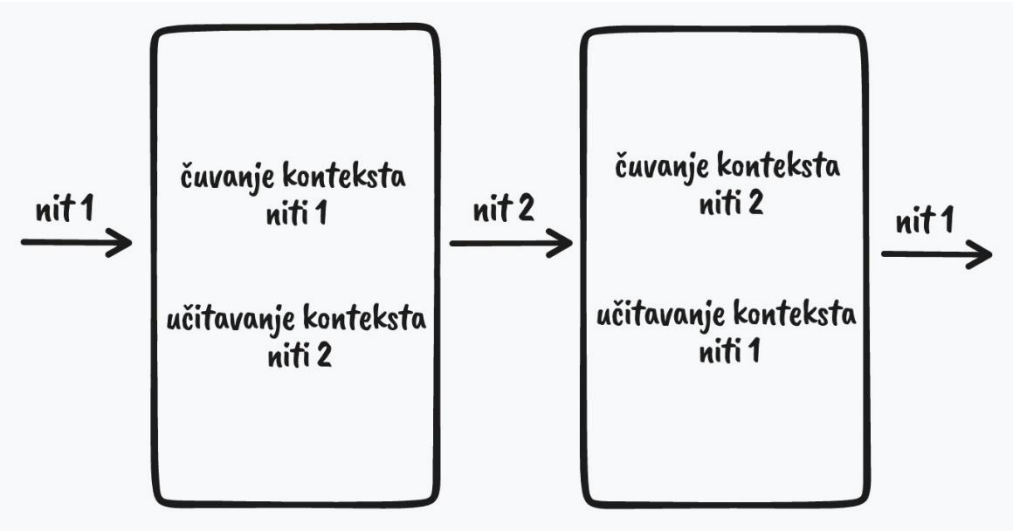
```
Your turn!
My turn!
My turn!
Your turn!
My turn!
My turn!
Your turn!
My turn!
My turn!
Your turn!
My turn!
My turn!
```

Slika 5 - Izvršavanje procesa sa nitima

Dakle, niti će sada naizmenično dobijati resurse za izvršavanje, pa će se isto tako ispisivati *My turn!* i *Your turn!*

Izvršavanje niti zasniva se na **promeni konteksta** (engl. *context switching*). Promenu konteksta ste do sada pominjali kada je reč o procesima. Jedan proces se izvršava na procesoru, ali u jednom trenutku je procesorsko vreme potrebno dodeliti drugom procesu. Tom prilikom se čuvaju podaci kontrolnog bloka jednog procesa, učitavaju se podaci kontrolnog bloka drugog procesa, pa on nastavlja sa izvršavanjem. Ovo se dešava uglavnom usled nekog sistemskog prekida ili sistemskog poziva.

Slična stvar se dešava i kada se niti izvršavaju **konkurentno**. Naime, kontekst niti koja se trenutno izvršava se čuva, a učitava se kontekst druge niti, kojoj će sada biti dodeljeni resursi za izvršavanje. Pod pojmom „kontekst niti'" podrazumevaju se podaci o vrednostima registara, brojaču naredbi, sadržaj steka, stanje niti i drugo. Promena konteksta omogućava da zamenjena nit kasnije nastavi sa izvršavanjem, tako što će se sačuvane vrednosti konteksta te niti ponovo učitati.



Slika 6 - Promena konteksta

Kada su u pitanju niti, promena konteksta se dešava kada se jedna nit izvrši, te se sada može izvršavati neka druga, nit dobije resurs na koji je čekala, pa je sada njeno izvršavanje moguće, ili, jednostavno, procesor, po nekom algoritmu (npr. *round robin*, *shortest job first* itd.) odluči da je vreme da se jedna nit prekine kako bi se izvršavala druga, što omogućava *fer* raspodelu procesorskog vremena između niti.

4. Utrkivanje (Race condition)

Posmatrajmo sledeći C program:

```
#include <stdio.h>
#include <pthread.h>

int vrednost;
void* saberi(void* arg);

int main(){

vrednost = 0;

pthread_t nit1, nit2;

pthread_create(&nit1, NULL, saberi, NULL);
pthread_create(&nit2, NULL, saberi, NULL);


pthread_join(nit1, NULL);
pthread_join(nit2, NULL);

printf("Vrednost: %d\n", vrednost);
return 0;
}

void* saberi (void* arg){
    int i = 0;

    for(i=0; i<100000; i++){
        vrednost++;
    }

}
```

Slika 7 – Program bez mutex-a (noMutex.c)

Inicijalizovane su dve niti i obema je dodeljen isti zadatak – da izvrše funkciju **`saberi`**. U ovoj funkciji se globalna promenljiva **`vrednost`** inkrementuje 100000 puta. Pošto niti dele **data** sekciju procesa, u kojoj se nalaze globalne promenljive, obe niti će pristupati promenljivoj **`vrednost`** i uvećavati je. Na prvi pogled, možemo zaključiti da će se, prilikom izvršavanja,

na terminalu ispisati vrednost 200000, odnosno, nakon što obe niti urade posao, vrednost promenljive **`vrednost`** bi trebalo da bude 200000. Međutim, to se neće uvek desiti:

```
[student@fedora mutex]$ ./noMutex
Vrednost: 90863
[student@fedora mutex]$ ./noMutex
Vrednost: 200000
[student@fedora mutex]$ ./noMutex
Vrednost: 200000
[student@fedora mutex]$ ./noMutex
Vrednost: 200000
[student@fedora mutex]$ ./noMutex
Vrednost: 200000
[student@fedora mutex]$ ./noMutex
Vrednost: 168693
```

Slika 8 - Izvršavanje programa sa slike 7

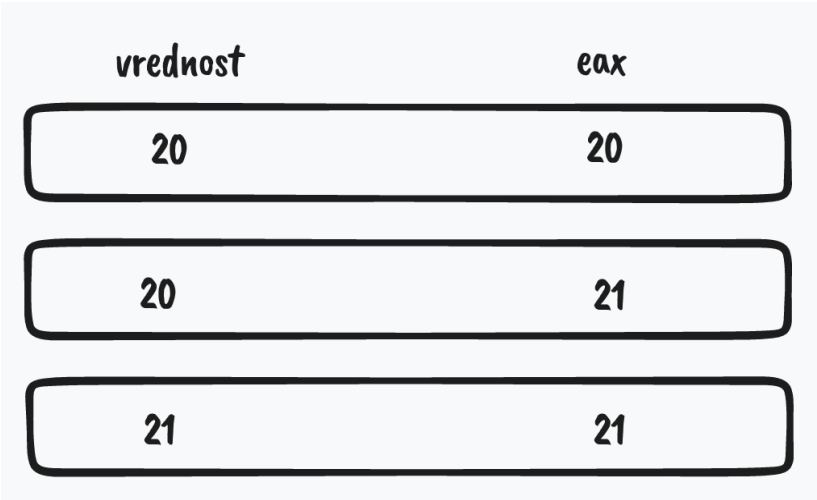
Dakle, uočavamo da dobijamo različite vrednosti kao rezultat izvršavanja ovog procesa. Šta se zapravo desilo, objasnićemo nakon što ovaj S program prevedemo na nivo asemblerskog koda (arhitektura x86-64), pomoću naredbe **`gcc -S noMutex.c`** (*noMutex.c* je naziv C programa na Slici 7). Zatim ćemo otvoriti novonastali fajl ***noMutex.s*** i doći do sekcije koda koja predstavlja liniju koda **`vrednost ++.`**

```
movl    vrednost(%rip), %eax
addl    $1, %eax
movl    %eax, vrednost(%rip)
```

Slika 9 - Linija vrednost++ prevedena u asemblerski kod

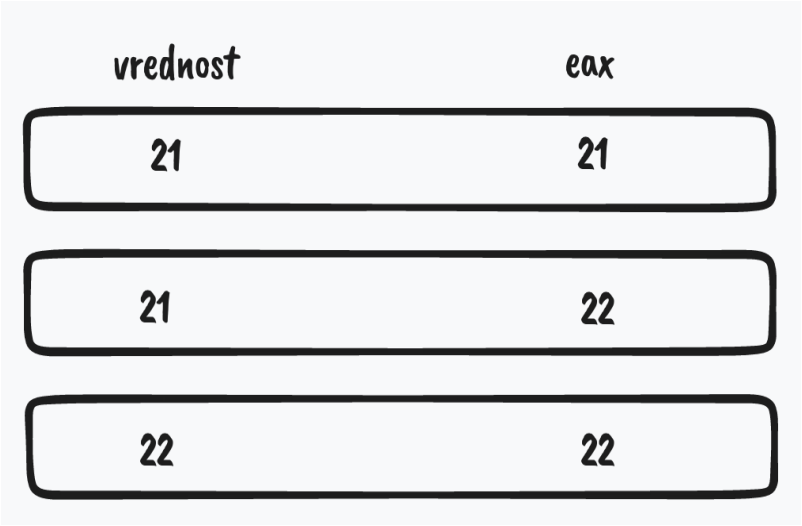
Možemo zaključiti da se ono što je nama predstavljeno kao atomska operacija u C programskom jeziku, linija **`vrednost++`**, zapravo predstavlja kao ove tri instrukcije (Slika 9) u asembleru.

Ono što se dešava u idealnom slučaju jeste da nit dođe do ove sekcije koda (od sada ćemo je zvati **kritična sekcija**) i izvrši sve tri instrukcije. Kritična sekcija podrazumeva deo koda u kom se pristupa deljenim resursima (u ovom slučaju deljeni resurs je promenljiva **`vrednost`**). Neka je u datom trenutku vrednost promenljive **`vrednost`** jednaka 20.



Slika 10 - Prva nit izvršava inkrementaciju

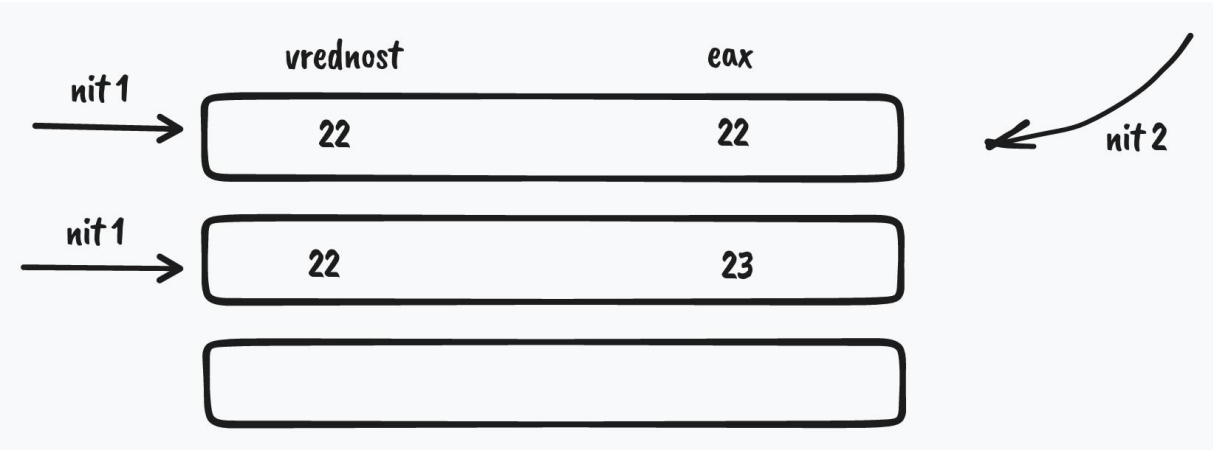
U prvoj liniji se vrednost 20 upisuje u registar **`eax`**. U drugoj liniji se vrši inkrementacija – vrednost u registru **`eax`** se uvećava za 1. U trećoj liniji se vrednost iz registra **`eax`** upisuje u promenljivu **`vrednost`**. Nakon ove tri instrukcije, vrednost promenljive **`vrednost`** je 21. Ukoliko i druga nit na isti način pristupi ovom zadatku, vrednost promenljive **`vrednost`** će naposletku biti 22.



Slika 11 - Druga nit izvršava inkrementaciju

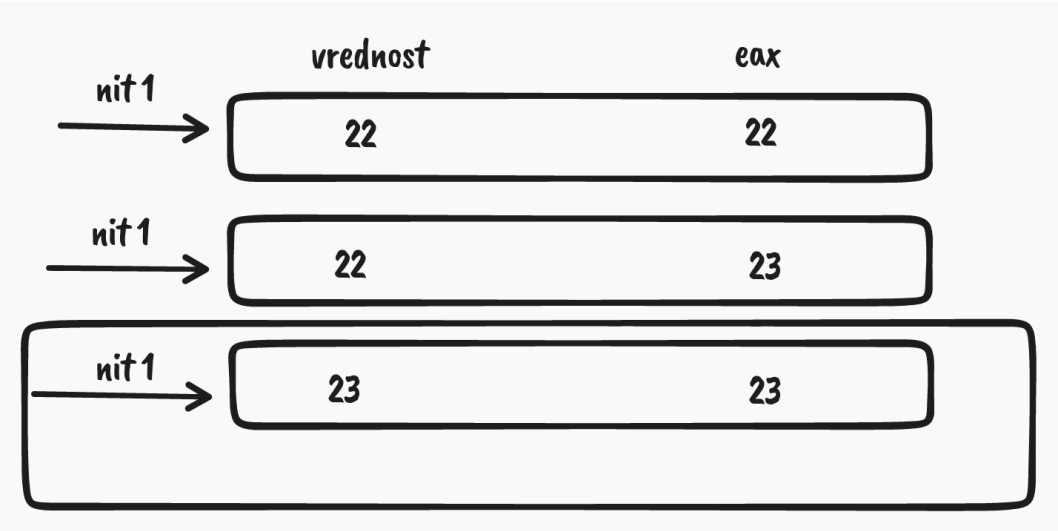
Iako je pomenuti način izvršavanja potpuno realan, on se neće desiti u nekom broju slučajeva, što smo i zaključili pogledavši Sliku 8. Kada bi se svaka iteracija odigrala na ovaj, idealan način, krajnji rezultat bi bio 200000, što se nije dogodilo. Ono što se može desiti je sledeće:

Prva nit dolazi do kritične sekcije koda. Upisuje vrednost 22 u **`eax`**, a zatim vrši inkrementaciju vrednosti koja se nalazi u registru **`eax`**. Međutim, u tom trenutku se procesorsko vreme dodeljuje drugoj niti.



Slika 12 - Procesorsko vreme se dodeljuje niti 2

Druga nit kreće da izvršava kritičnu sekciju koda od prve linije. Vrednost promenljive **`vrednost`** je i dalje 22 (primetite da prva nit nije uspela da upiše vrednost 23 nazad u promenljivu). Druga nit upisuje vrednost 22 u **`eax`**, uvećava vrednost na 23 i upisuje vrednost 23 nazad u promenljivu **`vrednost`**. Druga nit je time završila zadatak, pa se procesorsko vreme ponovo dodeljuje prvoj niti. Učitava se kontekst prve niti i ona treba da nastavi tamo gde je stala pre nego što je prekinuta, odnosno, treba da upiše inkrementovanu vrednost nazad u promenljivu **`vrednost`**. Ta vrednost je takođe 23. Nakon što su obe niti izvršile inkrementaciju, vrednost promenljive **`vrednost`** je 23, umesto očekivanog 24.



Slika 13 - Procesorsko vreme se ponovo dodeljuje niti 1 i ona nastavlja tamo gde je stala

Ovaj problem se zove **utrkivanje** (engl. *race condition*) i predstavlja situaciju kada više niti ili procesa pristupa nekom deljenom resursu (čita ili upisuje), a krajnji rezultat zavisi od redosleda kojim niti pristupaju tom deljenom resursu.

U našem slučaju se problem ogleda u tome što nit uspeva da pristupi kritičnoj sekciji koda koju u istom trenutku već izvršava neka druga nit. Ovaj problem se može rešiti **mehanizmom međusobnog isključivanja** (engl. *mutual exclusion*) koji omogućava da kada jedna nit pristupi kritičnoj sekciji koda, ni jedna druga nit ne može pristupiti toj sekciji, sve dok prva nit ne izvrši posao u celosti. U našem slučaju je potrebno omogućiti da kada jedna nit krene da izvršava naredbu **`vrednost++`**, odnosno, tri instrukcije u asemblerskom kodu, druga nit **ne može pristupiti toj sekciji koda sve dok prva nit ne izvrši sve tri instrukcije**. Za rešavanje ovog problema koristićemo **mutex mehanizam zaključavanja** i to u obliku promenljive tipa **`pthread_mutex_t`**.

```c
#include <stdio.h>
#include <pthread.h>

int vrednost;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
void* saberi(void* arg);

int main(){

vrednost = 0;

pthread_t nit1, nit2;

pthread_create(&nit1, NULL, saberi, NULL);
pthread_create(&nit2, NULL, saberi, NULL);


pthread_join(nit1, NULL);
pthread_join(nit2, NULL);

printf("Vrednost: %d\n", vrednost);
return 0;
}

void* saberi (void* arg){
    int i = 0;

    for(i=0; i<1000000; i++){
        pthread_mutex_lock(&mutex);
        vrednost++;
        pthread_mutex_unlock(&mutex);
    }

}
```

Slika 14 - Program sa mutex-om (mutex.c)

Nit koja pristupa kritičnoj sekciji koda treba da **zaključa mutex**. Zaključan mutex nagoveštava da je jedna nit pristupila kritičnoj sekciji koda i izvršava je, te druga nit trenutno ne može pristupiti toj sekciji koda. Druga nit će moći da pristupi kritičnoj sekciji koda tek kada prva nit, pri izlasku iz kritične sekcije, **ponovo otključa muteks**. Ono što je važno je da **nit koja je zaključala muteks, mora i da ga otključa**, kako bi sledeća nit mogla da pristupi kritičnoj sekciji koda. U liniji koda

```c
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

inicijalizujemo promenljivu tipa mutex. Obratite pažnju na kritičnu sekciju koda:

```
pthread_mutex_lock(&mutex);
vrednost++;
pthread_mutex_unlock(&mutex);
```

Slika 15 - Kritična sekcija koda

Dakle, pre kritične sekcije koda, nit treba da zaključa mutex, a po izlasku da ga otključa, kako bi sledeća nit mogla da pristupi ovoj sekciji koda na isti način. Kao što i samo ime funkcija govori, funkcija **pthread_mutex_lock** zaključava mutex, dok ga **pthread_mutex_unlock** otključava. Obema funkcijama se prosleđuje adresa mutex-a.

Na ovaj način smo rešili problem utrkivanja niti i kada kompajliramo i, ma koliko puta, pokrenemo ovaj program, uvek ćemo dobiti isti rezultat: 200000.

```
[student@fedora mutex]$ ./mutex
Vrednost: 2000000
[student@fedora mutex]$ ./mutex
Vrednost: 2000000
[student@fedora mutex]$ ./mutex
Vrednost: 2000000
[student@fedora mutex]$ ./mutex
Vrednost: 2000000
```

Slika 16 - Izvršavanje nakon uvođenja mutex-a

Ostalo je još da objasnimo funkciju **pthread_join**. Ova funkcija osigurava da glavna nit procesa (ona koja poziva funkciju **pthread_create**) neće biti završena pre nego što se izvrše kreirane niti. Dakle, ova funkcija omogućava da se sačeka izvršenje niti (**nit1** i **nit2**) pre nego što se ceo proces završi. U suprotnom bi se, u nekom broju slučajeva, glavna nit završila pre niti **nit1** u **nit2**, pa bi se prekinulo i njihovo izvršavanje pre nego što uspeju da obave svoje zadatke. Funkcija **pthread_join** kao argument prima nit koju je potrebno sačekati da se izvrši i promenljivu tipa **void **** u kojoj bi se uskladištila povratna vrednost niti. Pošto nas u ovom slučaju ne interesuje povratna vrednost niti, kao drugi argument prosleđujemo **NULL**.

Pored utrkivanja, postoje i drugi problemi koji se javljaju u radu sa nitima (i procesima), a to su kružno blokiranje i izgladnjivanje (engl. *starvation*).

Kružno blokiranje se javlja, recimo, kada imamo dve niti T1 i T2 (ili dva procesa) i dva resursa R1 i R2. Da bi niti izvršile svoje zadatke, moraju imati vlasništvo nad oba resursa istovremeno. Ono što se može desiti je da nit T1 dobije vlasništvo nad resursom R2, a T2 nad resursom R1. Niti T1 je potreban resurs R1 da bi izvršila zadatak, a niti T2 je potreban resurs R2. Obe niti su blokirane jer čekaju da dobiju resurs koji se nalazi u posedstvu druge niti. Ni jedna nit neće osloboditi resurs jer nije obavila zadatak. Niti T1 i T2 se nalaze u kružnoj blokadi.

Izgladnjivanje (engl. *starvation*) je situacija u kojoj jedna od niti (ili procesa) nikad ne dolazi na red za izvršavanje i to bez kružnog blokiranja. Tri niti T1, T2 i T3 zahtevaju pristup deljenom resursu R da bi izvršile zadatak. Recimo da je na početku resurs u vlasništvu niti T1, a zatim prelazi u vlasništvo niti T3. U međuvremenu T1 ponovo traži i dobija pristup, a zatim ponovo T3 i ovo se ponavlja naizmenično. Niti T2 je dugoročno onemogućen pristup resursu iako ne postoji problem kružnog blokiranja

5. Interprocesna komunikacija

Interprocesna komunikacija je mehanizam koji nam pruža operativni sistem i koji omogućava komunikaciju između procesa. Pod komunikacijom podrazumevamo razmenu podataka od jednog procesa do drugog ili signaliranje procesa o izvršenju određenog događaja.

Postoji nekoliko pristupa u implementaciji interprocesne komunikacije:

- **Pipe** – pipe je jednosmerni kanal za prenos podataka. Uz pomoć dva pipe-a moguće je uspostaviti dvosmernu komunikaciju između dva procesa. Pipe mehanizam zasniva se na upotrebi standardnih tokova (engl. *standard streams*), gde se komunikacija ostvaruje tako što se standardni izlaz jednog procesa prosleđuje na standardni ulaz drugog procesa. Razlikujemo dva tipa *pipe*-ova: **neimenovani** i **imenovani**. Neimenovani *pipe* upisuje podatke izlaznog toka procesa u bafer koji će biti pročitan od strane drugog procesa, nakon čega će ovaj jednosmerni kanal komunikacije nestati. Kod imenovanog *pipe*-a se podaci jednog procesa upisuju u fajl koji ostaje na računaru i nakon što procesi prestanu sa radom. Sintaksa za neimenovane *pipe*-ove na *Unix* sistemima je sledeća:

  ```
  naredba1 | naredba2
  ```

  Sistemski poziv za kreiranje neimenovanog pipe-a jeste `pipe()`, dok se za kreiranje imenovanih pipe-ova koriste sistemski pozivi `mkfifo()` ili `mknod()`.
- **Soket** (engl. *socket*) – soket je krajnja tačka za slanje ili primanje podataka putem mreže. Postoje dva tipa soketa: **soketi za interprocesnu komunikaciju** i **soketi za mrežnu komunikaciju**. Soketi za interprocesnu komunikaciju omogućavaju komunikaciju između procesa na istom računaru (hostu), dok mrežni soketi dozvoljavaju da procesi budu pokrenuti na različitim računarima.
- **Signali** (engl. *signals*) – jeste mehanizam interprocesne komunikacije pomoću prekida (engl. *interrupts*). Signali mogu nastati kroz korisničku interakciju. Na primer, kada korisnik pritisne kombinaciju tastera Ctrl+C kako bi terminirao aktivni proces u terminalu, generiše se SIGTERM signal. Signali mogu nastati i od strane drugog procesa i na taj način se ostvaruje interprocesna komunikacija. Tako, na primer, jedan proces može da pozove funkciju `kill` kojoj će proslediti PID (Process ID) procesa koji hoće da terminira.

- **Red sa porukama** (engl. *message queue*) – mehanizam interprocesne komunikacije koji omogućava procesima da razmenjuju podatke u vidu poruka. Predstavlja asinhroni način komunikacije u kojem se poruke smeštaju u red, čekajući da budu procesuirane, nakon čega se izbacuju iz reda. Četiri bitne funkcije za uspostavljanje reda sa porukama jesu:

  `msgget` za kreiranje i pristup redu sa porukama, **`msgsnd`** za dodavanje poruka u red, **`msgrcv`** za dobijanje poruka iz reda i **`msgctl`** koja predstavlja kontrolnu funkciju za izvršenje kontrolnih operacija
- **Deljena memorija** (engl. *shared memory*) – mehanizam deljene memorije omogućava dvama procesima da dele isti segment fizičke memorije. Pošto se deljeni memorijski segment mapira na adresni prostor svakog od procesa, nije potrebna intervencija kernela za samu komunikaciju između procesa. Sve što je potrebno jeste da proces upiše podatke u mapirane adrese i podaci će biti dostupni svim procesima koji koriste deljeni memorijski segment. Ovaj mehanizam je zbog toga brži u odnosu na pipe ili red sa porukama, pošto kod njih proces koji šalje podatke mora prvo da ih iskopira iz korisničkog prostora u prostor kernela, pri čemu proces koji prima podatke mora nakon toga da ih iskopira u obrnutom redosledu. Bitno je napomenuti da se memorijski segment ne mora mapirati na iste adrese kod različitih procesa. Za uspostavljanje interprocesne komunikacije mogu se koristiti funkcije iz ***System V ABI***-ja: **`shmget`** za kreiranje novog ili dobijanje identifikatora postojećeg segmenta, **`shmat`** za mapiranje memorijskog segmenta na adresni prostor procesa, **`shmdt`** za oslobađanje dealociranje zauzete memorije u adresnom prostoru procesa i **`shmctl`** za brisanje deljenog memorijskog segmenta.

5.1. Mehanizam deljene memorije

Mehanizam deljene memorije biće prikazan na problemu čitalaca i pisaca (engl. *readers-writers problem*). U problemu su opisana dva procesa koji pristupaju jednom zapisu (deljenom resursu). Proces pisca generiše podatke i smešta ih u zapis, a proces čitaoca uzima jedan po jedan podatak iz zapisa i čita ga. Problem se sastoji u tome da treba obezbediti da nijedna nit ne sme da vrši ni čitanje ni upis, ako u isto vreme druga nit vrši operaciju upisa u isti deljeni resurs. U ovoj sekciji se neće odraditi sam problem sinhronizacije, već će on biti rešen u narednoj sekciji, dok je u ovoj akcenat na uspostavljanju deljenog memorijskog segmenta između procesa.

Napravićemo dva C programa, **write.c** koji će imati ulogu pisca i **read.c** koji će imati ulogu čitaoca. Za uspostavljanje interprocesne komunikacije korićenjem mehanizma deljene memorije koristiće se ***System V ABI***. ***Application Binary Interface*** (ABI) je interfejs između dva binarna modula programa. Jedan od tih modula je često biblioteka, dok je drugi korisnički program. ABI definiše kako se strukturama podataka i funkcijama pristupa na nivou mašinskog koda, za razliku od API-ja koji definiše funkcije kojima je moguće pristupiti na nivou izvornog koda. Tako na primer, jedna od stvari koju ABI definiše jeste konvencija poziva funkcija koja određuje kako se funkcija poziva u mašinskom kodu, kako se argumenti prosleđuju funkciji i gde se čuva izlazna vrednost funkcije.

Prvo je potrebno da uključimo potrebne biblioteke u naš program write.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdbool.h>
```

Slika 17 - Biblioteke

Osim delova standardne biblioteke, uključili smo i biblioteke **`sys/types.h`**, **`sys/ipc.h`** i **`sys/shm.h`** koje su nam potrebne za uspostavljanje interprocesne komunikacije i rad sa

deljenom memorijom. Da bismo uspostavili komunikaciju pomoću deljene memorije potrebno je da odradimo nekoliko koraka:

- Potrebno je da kreiramo ili dobijemo postojeći deljeni memorijski segment na osnovu ključa koji ćemo generisati
- Potrebno je da mapiramo deljeni memorijski segment na adresni prostor procesa
- U memorijski segment upisujemo odgovarajuće podatke ili čitamo iz njega
- Nakon završetka rada potrebno da oslobodimo adrese koje je zauzeo deljeni memorijski segment

5.2.1. Dobijanje deljenog memorijskog segmenta

Za dobijanje deljenog memorijskog segmenta koristićemo funkciju

```
int shmget(key_t key, size_t size, int shmflg);g
```

koja u zavisnosti od prosleđenih argumenata vraća identifikator deljenog memorijskog segmenta. Funkcija kao izlaznu vrednost vraća identifikator deljenog memorijskog segmenta. Kao prvi parametar, funkcija prima vrednost tipa **`key_t`** i ova vrednost predstavlja ključ koji će koristiti svi procesi koji učestvuju u interprocesnoj komunikaciji kako bi dobili isti deljeni memorijski segment. Kao drugi argument prosleđuje se veličina memorijskog segmenta i ova veličina se zaokružuje na najbližu veličinu deljivu sa veličinom stranice. Treći parametar definiše opcije koje se koriste pri dobijanju deljenog segmenta kao i dozvole koje svaki od korisnika ima nad memorijskim segmentom. Donjih devet bitova flag-a definiše dozvole koje svaki korisnik ima nad memorijskim segmentom i definiše se kao oktalno vrednost. Preostali bitovi predstavljaju opcije pri kreiranju segmenta. Dve bitne vrednosti ovde jesu **`IPC_CREAT`** i **`IPC_EXCL`**. Ukoliko se prosledi samo **`IPC_CREAT`** kao vrednost parametra **`shmflg`**, kreiraće se novi memorijski segment i vratiće se njegov identifikator u slučaju da ne postoji segment asociran sa prosleđenim ključem, u suprotnom vratiće se identifikator postojećeg memorijskog segmenta. Ukoliko se opcija **`IPC_CREAT`** ukombinuje sa opcijom **`IPC_EXCL`** deljeni segment će se kreirati i vratiće se njegov identifikator ukoliko ne postoji segment asociran sa prosleđenim ključem, u suprotnom funkcija će vratiti grešku.

Za kreiranje ključa koristimo funkciju

```
key_t ftok(char *pathname, int proj_id);
```

Ova funkcija sadrži dva prametra: putanju do fajla i id. Funkcija na osnovu ova dva parametra generiše ključ koji će biti isti svaki put kad se funkcija pozove sa istom kombinacijom

argumenata. Ukoliko je došlo do greške prilikom generisanja ključa, koja može nastati na primer ukoliko ne postoji prosleđeni fajl, ili nemamo dozvole da otvorimo fajl, funkcija će vratiti vrednost -1. Pošto je izlazna vrednost -1, koja predstavlja grešku, zajednička za većinu funkcija koje ćemo koristiti, ovu vrednost definisaćemo kao makro pre definisanja samih funkcija:

```
#define IPC_RESULT_ERROR -1
```

Slika 18 - Makro za greške

Funkcija za generisanje ključa i dobijanje deljenog memorijskog segmenta data je u nastavku:

```
int get_shared_block(char *filename, int size){

    key_t key;

    key = ftok(filename, 0);

    if(key == IPC_RESULT_ERROR){
        return IPC_RESULT_ERROR;
    }

    return shmget(key,size, 0644 | IPC_CREAT);
}
```

Slika 19 - Funkcija za generisanje ključa i dobijanje segmenata

Mapiranje deljenog memorijskog segmenta na adresni prostor procesa

Nakon što smo dobili deljeni memorijski segment, potrebno je taj segment pridružiti adresnom prostoru svakog od procesa. Za to koristimo funkciju:

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

Funkciji prosleđujemo id memorijskog segmenta, adresu na koju će da se mapira dati segment i određeni flag. Funkcija nam vraća početnu adresu memorijskog segmenta. Ukoliko ne želimo da specificiramo adresu od koje će mapiranje započeti, možemo kao drugi argument proslediti NULL, a kao flag vrednost 0. Funkcija za mapiranje deljenog memorijskog segmenta data je u nastavku:

```
26 char * attach_memory_block(char *filename, int size){
27     int shared_block_id = get_shared_block(filename, size);
28     char *result;
29     result = shmat(shared_block_id, NULL, 0);
30     if(result == (char *)IPC_RESULT_ERROR){
31         return NULL;
32     }
33     return result;
34
35 }
36
```

Slika 20 - Mapiranje memorijskog segmenta

5.2.2. Upisivanje ili čitanje iz memorijskog segmenta

U **`main`** funkciji programa write.c argument komandne linije upisuje se u memorijski blok koristeći funkciju

```
char * strncpy ( char * destination, const char * source, size_t num
);
```

Funkcija **`strncpy`** upisuje *string* **`source`** u **`destination`** pri čemu se zadaje i veličina bloka kao treći parametar. Funkcija **`main`** programa write.c data je u nastavku:

```c
int main(int argc, char *argv[]){


    char *block = attach_memory_block(FILENAME, BLOCK_SIZE);

    if(block ==NULL){
        printf("Error! Unable to get shm block!");
        return -1;
    }

    printf("Writing: \"%s\"\n", argv[1]);
    strncpy(block, argv[1], BLOCK_SIZE);

    detach_memory_block(block);

    return 0;
}
```

Slika 21 - main funkcija programa write.c (bez semafora)

Ovde koristimo dva makroa: **`FILENAME`** i **`BLOCK_SIZE`**. Makro **`FILENAME`** jeste putanja do fajla koji se koristi u funkciji **`ftok`**, a **`BLOCK_SIZE`** predstavlja veličinu deljenog memorijskog segmenta:

```c
#define BLOCK_SIZE 4096
#define FILENAME "write.c"
```

Slika 22 - Potrebni makroi

Program read.c treba samo da pročita podatke koji su upisani u deljeni memorijski segment i njegova **`main`** funkcija je data u nastavku:

```c
int main(void){

    char *block = attach_memory_block(FILENAME, BLOCK_SIZE);
    if(block == NULL){
        printf("Error! Unable to get shm block!");
        return -1;
    }

    printf("Reading: \"%s\"\n", block);

    detach_memory_block(block);

    return 0;
}
```

Slika 23 - main funkcija programa read.c (bez semafora)

Na kraju **`main`** funkcine oba programa, poziva se funkcija **`detach_memory_block`** koja oslobađa adresni prostor procesa nakon završetka rada sa deljenim memorijskim segmentom. Ova funkcija je objašnjena u narednoj sekciji.

5.2.3. Oslobađanje adresnog prostora

Funkcija za oslobađanje memorijskih adresa koje je zauzeo deljeni memorijski segment je

`int shmdt(const void *shmaddr);`

Funkcija prima pokazivač ka deljenom memorijskom segmentu, a kao izlaz vraća 0 u slučaju da je uspešno oslobođen adresni prostor, a -1 u slučaju da je došlo do greške.

Funkcija **`detach_memory_block`** data je u nastavku:

```c
bool detach_memory_block(char *block){
    return (shmdt(block) != IPC_RESULT_ERROR);
}
```

Slika 24 - Funkcija detach_memory_block

6. Semafori

Prvi veliki napredak u rešavanju problema konkurentnih procesa dogodio se 1965. godine kada je holandski informatičar **Edsger Dajkstra** uveo koncept semafora. Ideja je sledeća: dva ili više procesa ili niti mogu se međusobno sinhronizovati korišćenjem jednostavnih signala, tako što bi određeni proces ne bi mogao da nastavi sa izvršavanjem od određene tačke sve dok ne dobije odgovarajući signal. Ovakav mehanizam sinhronizacije nam pruža struktura podataka *semafor*. Da bi se signal poslao od semafora s, poziva se funkcija **`post(s)`** (Dajkstra je ovoj funkciji dao naziv **`V`**), dok se za primanje signala koristi poziv **`wait(s)`** (Dajkstra je ovoj funkciji dao naziv **`P`**). Ukoliko signal nije još uvek poslat, proces (nit) koji očekuje signal se suspenduje sve dok se signal ne pošalje.

Da bi se postigao željeni efekat, semafori se posmatraju kao celobrojne promenljive nad kojima je moguće izvršiti tri operacije:

- Semafor je moguće inicijalizovati na nenegativan ceo broj
- Operacija **`wait`** smanjuje vrednost semafora za jedan. Ukoliko je vrednost semafora nakon ovoga negativna, proces (nit) koji je pozvao funkciju **`wait`** će preći u stanje blokiran. U suprotnom se normalno nastavlja izvršavanje programa.
- Operacija **`post`** povećava vrednost semafora za jedan. Ukoliko je vrednost semafora nakon ove operacije negativna ili jednaka nuli, proces koji je bio blokiran nakon operacije **`wait`** će biti odblokiran.

```
func V(Semaphore s){
   S++;
}

func P(Semaphore s){
  while (S<=0);

  S--;
}
```

Slika 25 - Prototip funkcija P (wait) i V (post)

Objašnjenje ovih operacija je sledeće. Semafor na početku ima ili vrednost 0 ili neku pozitivnu vrednost. Ukoliko je vrednost semafora pozitivna, ta vrednost zapravo predstavlja broj procesa (niti) koji mogu da pozovu operaciju **wait** bez da budu blokirani, odnosno broj procesa (niti) koji mogu istovremeno da se nađu u kritičnoj sekciji koda. Ukoliko je vrednost semafora nula, sledeći proces (nit) koji izvrši operaciju **wait** će biti blokiran. Svaki naredni poziv **wait** operacije će samo dodatno smanjiti negativnu vrednost semafora. Negativna vrednost semafora odgovara broju procesa (niti) koji čekaju da budu odblokirani. Svaki poziv operacije **post** dok je vrednost semafora negativna će odblokirati po jedan proces (nit).

Iz navedenog dobijamo tri posledice ovako definisanih semafora:

- Ne postoji način da proces (nit) zna pre poziva operacije **wait** da li će biti blokirana ili ne.
- Nakon što jedan proces (nit) uveća vrednost semafora i drugi proces (nit) bude odblokiran, oba procesa (niti) će nastaviti da se konkurentno izvršavaju. Nije moguće znati kojem procesu (niti) će biti dodeljeni resursi potrebni za izvršavanje kod jednoprocesorskih sistema.
- Kada se pozove funkcija post ne zna se da li neki proces čeka da bude odblokiran, tako da broj odblokiranih procesa može biti nula ili jedan.

U našem konkretnom primeru potrebno je uz pomoć semafora uvesti ograničenje da u jednom trenutku samo jedan proces može da izvršava svoj zadatak, odnosno da u trenutku kada **write** proces upisuje podatke u deljeni memorijski segment proces **read** ne može da čita podatke iz memorijskog segmenta, i obrnuto.

Za kreiranje semafora pomoću POSIX API-ja koristi se sledeća funkcija:

```
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
```

Funkciji prosleđujemo naziv semafora, opcije pri kreiranju semafora, dozvole nad semaforom i početnu vrednost semafora.

Operacije wait i post u okviru POSIX API-ja definisane su na sledeći način:

```
int sem_wait(sem_t *sem); int sem_post(sem_t *sem);
```

Za rad sa semaforima bitne su nam još dve funkcije:

```
int sem_unlink(const char *name); int sem_close(sem_t *sem);
```

Funkcija **`sem_unlink`** uklanja semafor sa prosleđenim imenom, ukoliko postoji. Funkcija **`sem_close`** uklanja prosleđeni semafor. U našem problemu, proces read će kreirati dva semafora, jedan za signaliranje pisca, a drugi za signaliranje čitaoca. Semafor čitaoca imaće početnu vrednost 1, dok će semafor pisca imati početnu vrednost nula, što znači da će proces pisca, pošto on čeka signal od semafora čitaoca čija je vrednost 1, moći da upiše vrednost u memorijski segment i da signalizira nakon toga procesu čitaoca da može da pročita upisanu vrednost.

Prvo je potrebno da uključimo dve dodatne biblioteke za rad sa semaforima:

```
8 #include <semaphore.h>
9 #include <fcntl.h>
```

Slika 26 - Biblioteke potrebne za rad sa semaforima

Nakon toga definišemo dva dodatna makroa koji će predstavljati nazive semafora:

```
15 #define SEM_PRODUCER_NAME "/myproducer"
16 #define SEM_CONSUMER_NAME "/myconsumer"
```

Slika 27 - Nazivi semafora

Ograničićemo da proces write deset puta upiše vrednost argumenta komandne linije, a da proces read čita vrednosti sve dok ne dobije poruku *quit*, nakon čega se proces read prekida.

Funkcija **`main`** za procese read i write data je u nastavku:

```c
int main(void){


char *block = attach_memory_block (FILENAME, BLOCK_SIZE);


    if(block == NULL){
        printf("Error! Unable to get shm block!" );
        return -1;
    }

    sem_unlink (SEM_CONSUMER_NAME );
    sem_unlink (SEM_PRODUCER_NAME );

    sem_t *sem_prod = sem_open(SEM_PRODUCER_NAME , O_CREAT, 0660, 0);

    sem_t *sem_cons = sem_open(SEM_CONSUMER_NAME , O_CREAT, 0660, 1);

    while(true){
        sem_wait(sem_prod);
        if(strlen(block)>0){
            printf("Reading: \"%s\"\n", block);
            bool done = (strcmp(block, "quit") == 0);
            block[0] = 0;
            if(done){
                break;
            }
        }
        sem_post(sem_cons);
    }
    sem_close(sem_cons);
    sem_close(sem_prod);
    detach_memory_block (block);

    return 0;
}
```

Slika 28 - main funkcija programa write.c (sa semaforima)

```c
int main(int argc, char *argv[]){


    char *block = attach_memory_block(FILENAME, BLOCK_SIZE);

    if(block ==NULL){
        printf("Error! Unable to get shm block!");
        return -1;
    }



    sem_t *sem_prod = sem_open(SEM_PRODUCER_NAME, 0);

    sem_t *sem_cons = sem_open(SEM_CONSUMER_NAME, 1);

    for(int i = 0; i< 10; i++){
        sem_wait(sem_cons);
        printf("Writing: \"%s\"\n", argv[1]);
        strncpy(block, argv[1], BLOCK_SIZE);
        sem_post(sem_prod);
    }

    sem_close(sem_prod);
    sem_close(sem_cons);
    detach_memory_block(block);

    return 0;
}
```

Slika 29 - main funkcija programa read.c (sa semaforima)

Proces read pokušava da smanji vrednost semafora **`sem_prod`**. Pošto je vrednost ovog semafora 0, proces read će biti blokiran sve dok ga proces write ne signalizira, odnosno ne poveća vrednost semafora **`sem_prod`**. Proces **`write`** pokušava da smanji vrednost semafora **`sem_cons`** i uspeva u tome pošto je njegova početna vrednost 1. Proces write nastavlja da izvršava svoje instrukcije i na kraju povećava vrednost semafora **`sem_prod`** čime proces read biva odblokiran. Ovaj proces se nastavlja sve dok proces write ne upiše sve vrednosti u deljeni memorijski segment ili dok ne upiše poruku **`quit`**.